

ARITHMETIC HARDWARE

Module.4

CDA 3103C

Mesut Ozdag, Ph.D.

Introduction

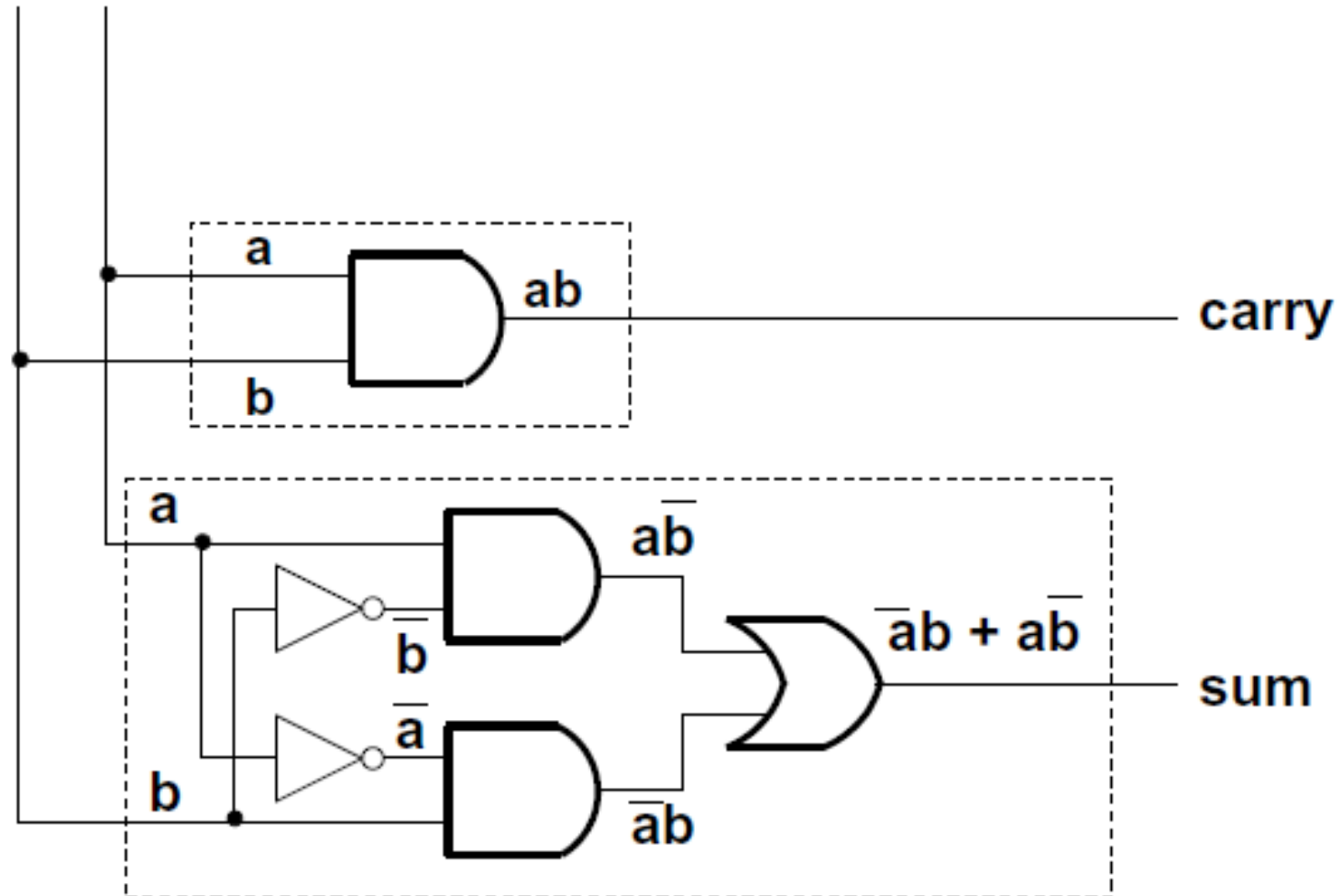
- Adders
 - Half
 - Full
 - Ripple-Carry
 - Carry-Lookahead
 - Carry-Select
- Arithmetic Logic Unit

1-Bit Half Adder

A	B	Sum	Cout
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

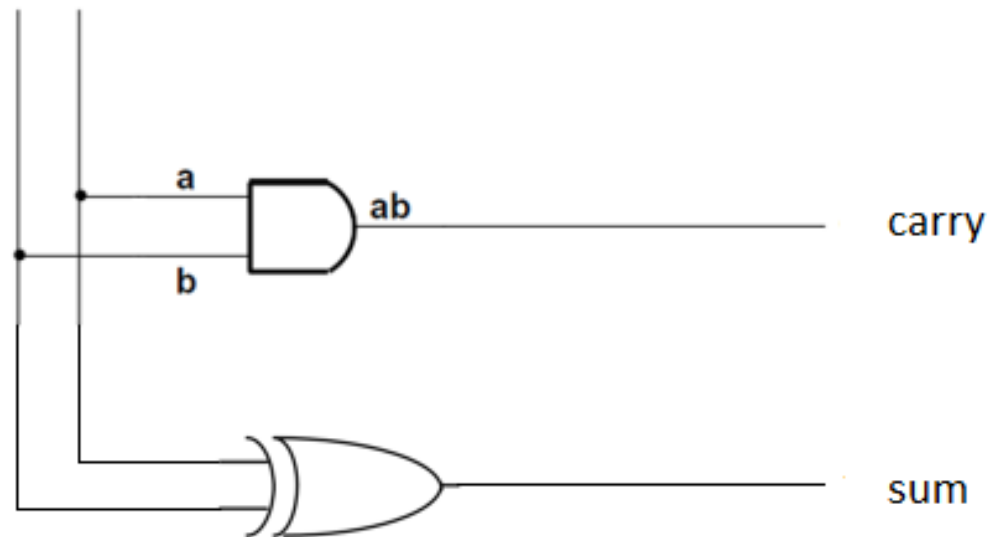
- $\text{Sum} = A'B + AB'$
- $\text{Cout} = AB$

1-Bit Half Adder



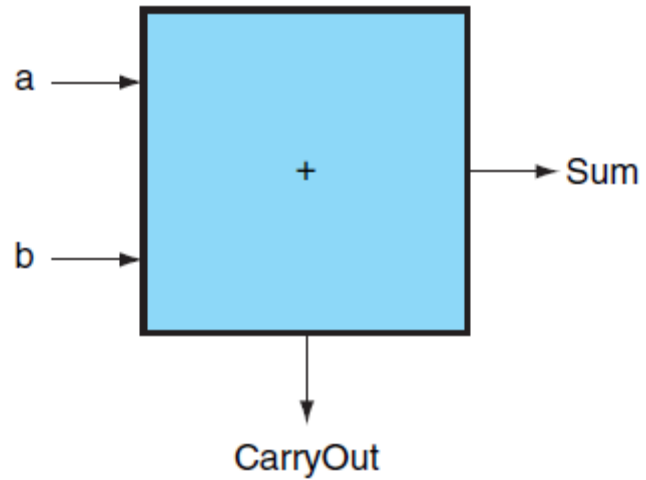
1-Bit Half Adder

- Sum = $A'B + AB'$
= A XOR B

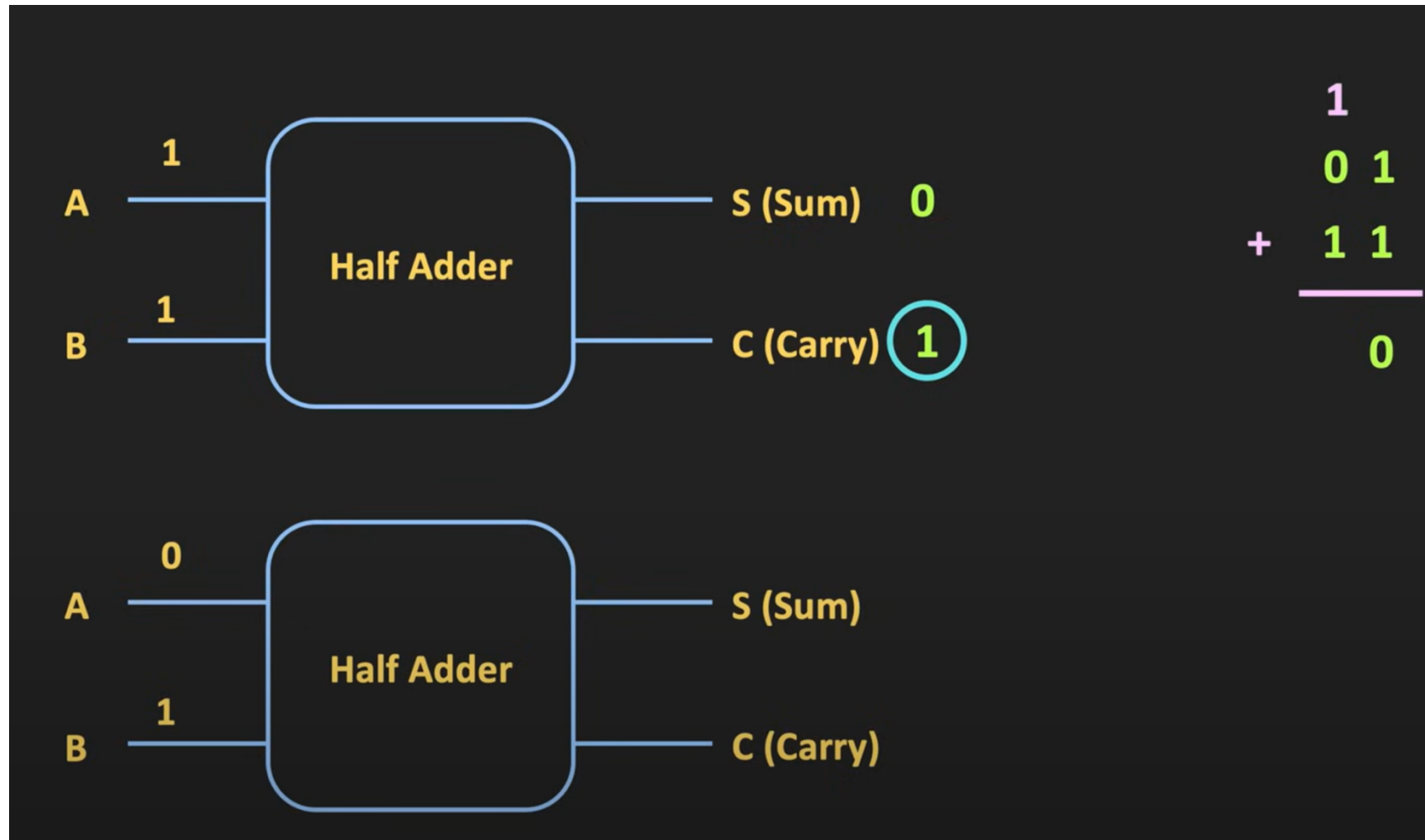


1-Bit Half Adder

- Block Diagram



What if there is an incoming carry from the previous addition?

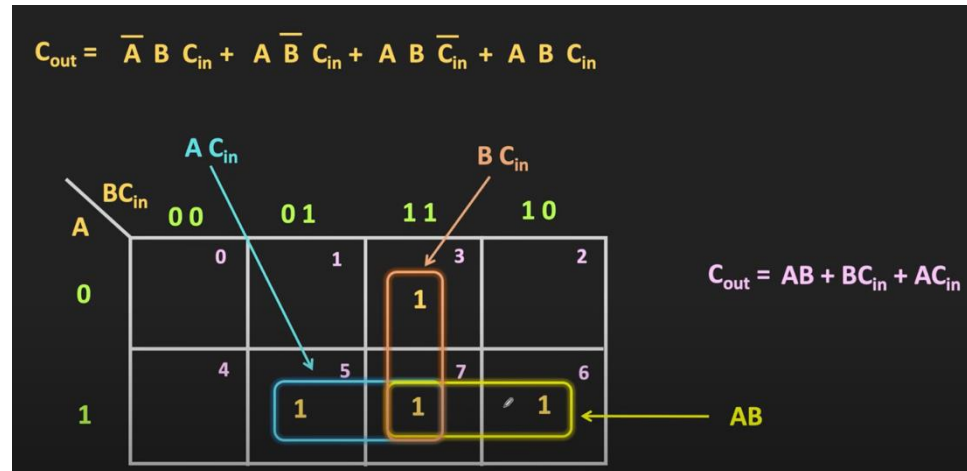


1-Bit Full Adder

- Sum = A XOR B XOR Cin
- Cout = AB + ACin + BCin

Inputs			Outputs		Comment
A	B	Cin	Cout	Sum	
0	0	0	0	0	0+0+0=00
0	0	1	0	1	0+0+1=01
0	1	0	0	1	0+1+0=01
0	1	1	1	0	0+1+1=10
1	0	0	0	1	1+0+0=01
1	0	1	1	0	1+0+1=10
1	1	0	1	0	1+1+0=10
1	1	1	1	1	1+1+1=11

$$\begin{aligned} S &= \bar{A} \bar{B} C_{in} + \bar{A} B \bar{C}_{in} + A \bar{B} \bar{C}_{in} + A B C_{in} \\ &= \bar{A} (\bar{B} C_{in} + B \bar{C}_{in}) + A (\bar{B} \bar{C}_{in} + B C_{in}) \\ &= \bar{A} (\underbrace{B \oplus C_{in}}_X) + A (\underbrace{\bar{B} \oplus \bar{C}_{in}}_X) \\ S &= A \oplus B \oplus C_{in} \end{aligned}$$



1-Bit Full Adder

- Standard approach
 - 6 gates
 - 2 XORs, 2 ANDs, 2 ORs

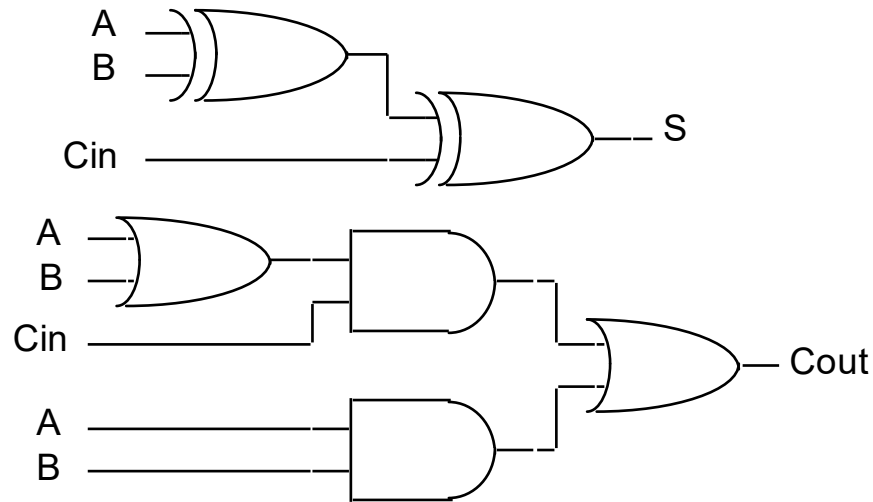
$$C_{out} = \overline{A} B C_{in} + A \overline{B} C_{in} + A B \overline{C}_{in} + A B C_{in}$$

$$= AB(C_{in} + \overline{C}_{in}) + C_{in}(\overline{A}B + A\overline{B})$$

$$= AB + C_{in}(A \oplus B)$$

$$C_{out} = AB + BC_{in} + AC_{in}$$

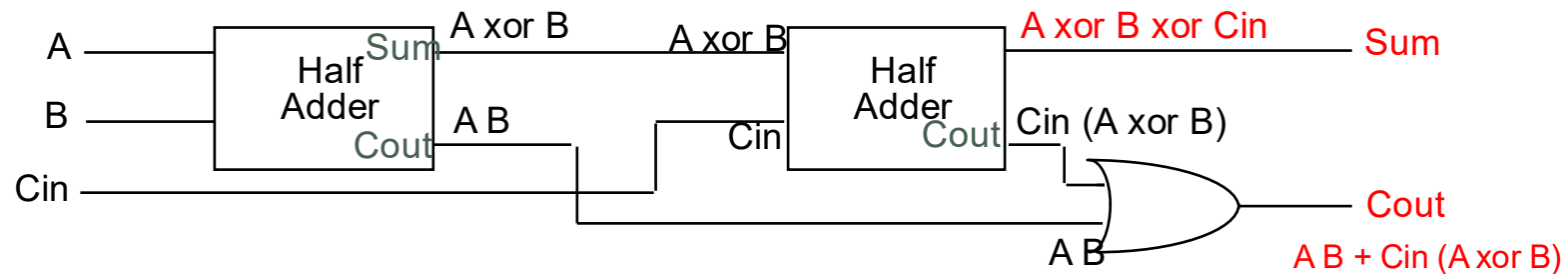
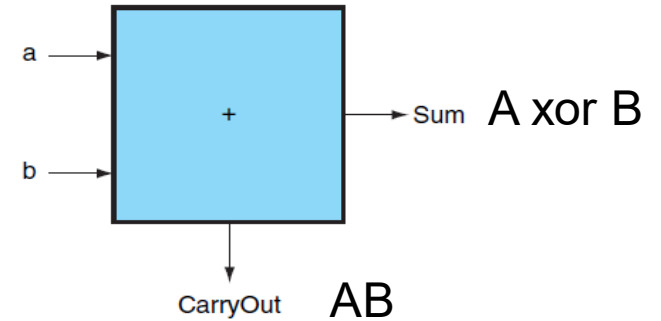
$$S = A \oplus B \oplus C_{in}$$



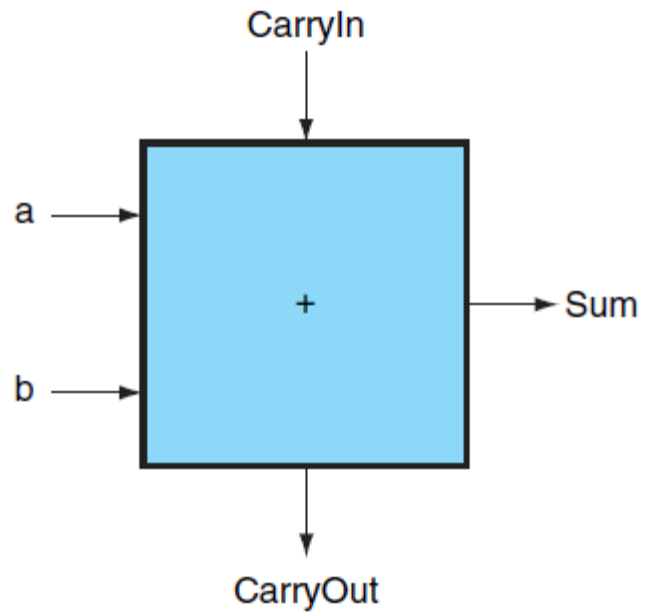
$$Cout = AB + Cin(A \oplus B) = AB + BCin + ACin$$

1-Bit Full Adder

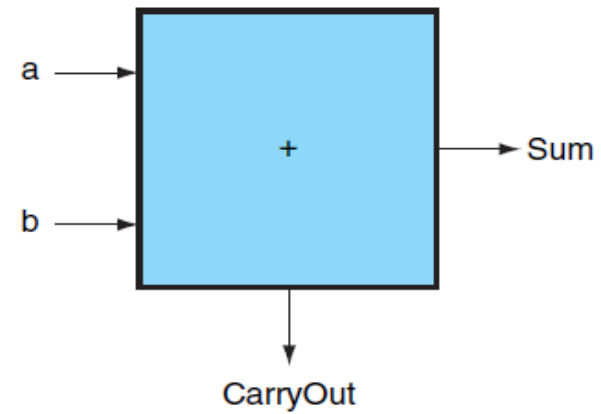
- Alternative implementation
 - 5 gates
 - half adder is an XOR gate and AND gate
 - 2 XORs, 2 ANDs, 1 OR



1-Bit Full Adder

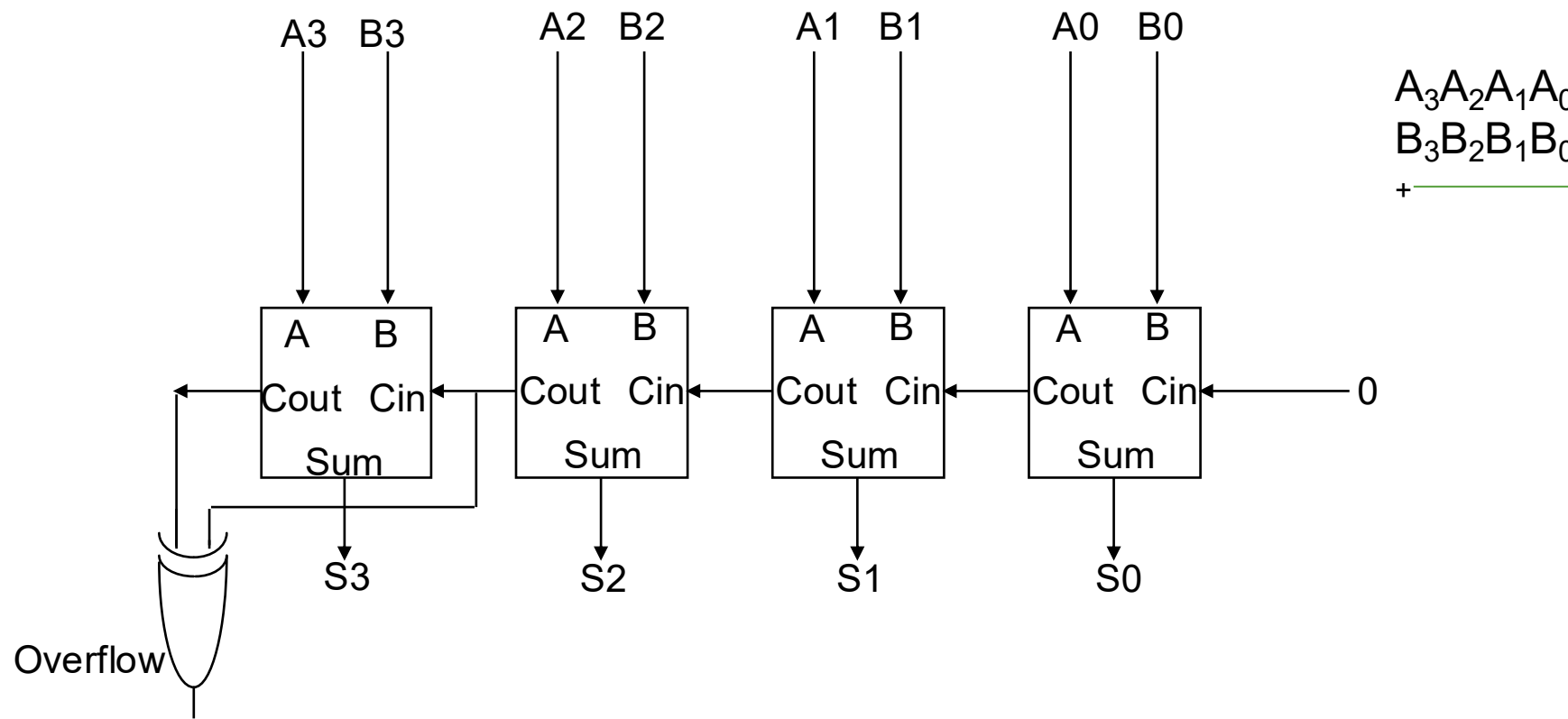


1-Bit Half Adder



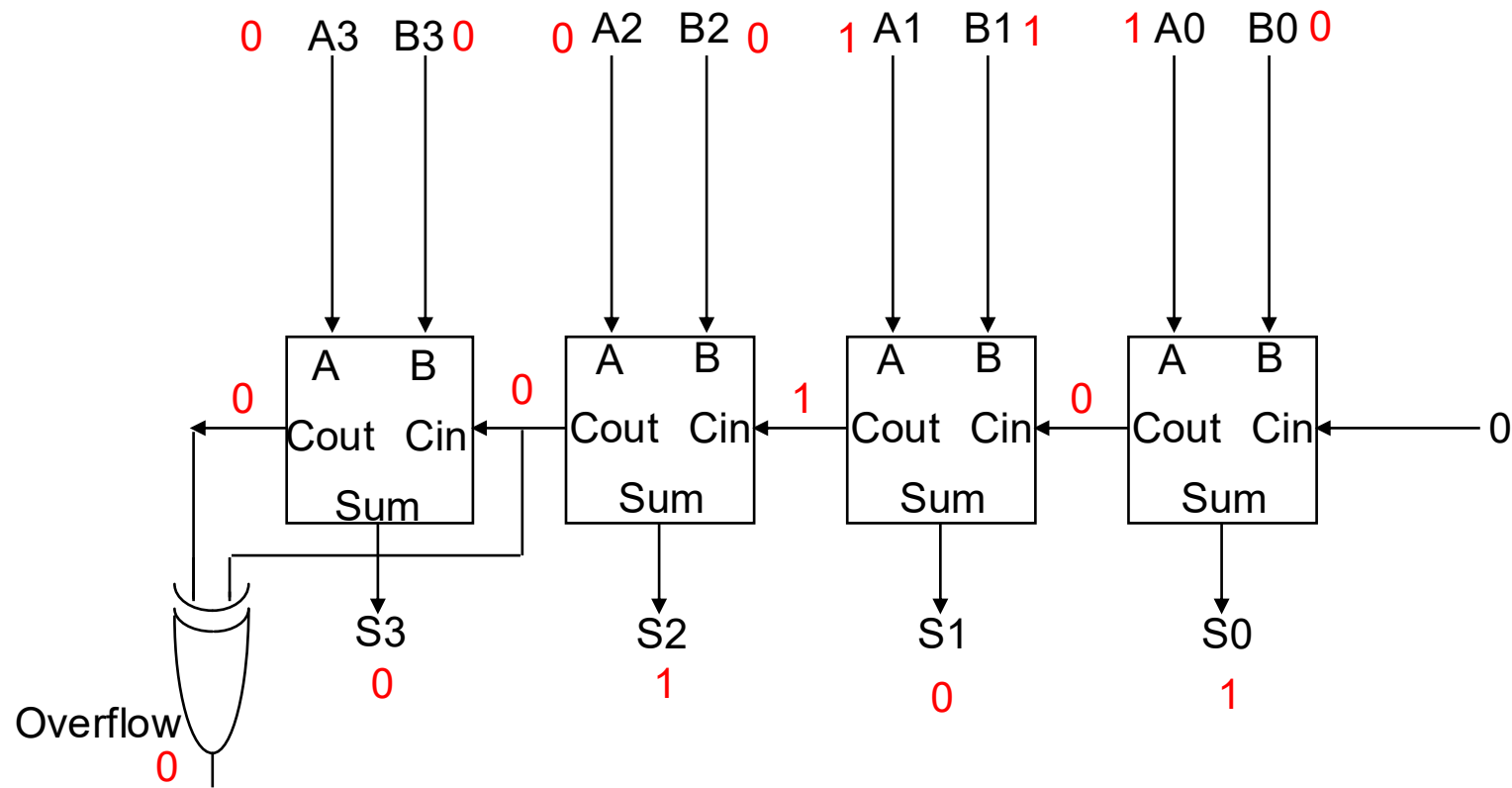
Ripple-Carry Adder

- To add binary numbers of lengths greater than 1, we cascade 1-bit adders together.



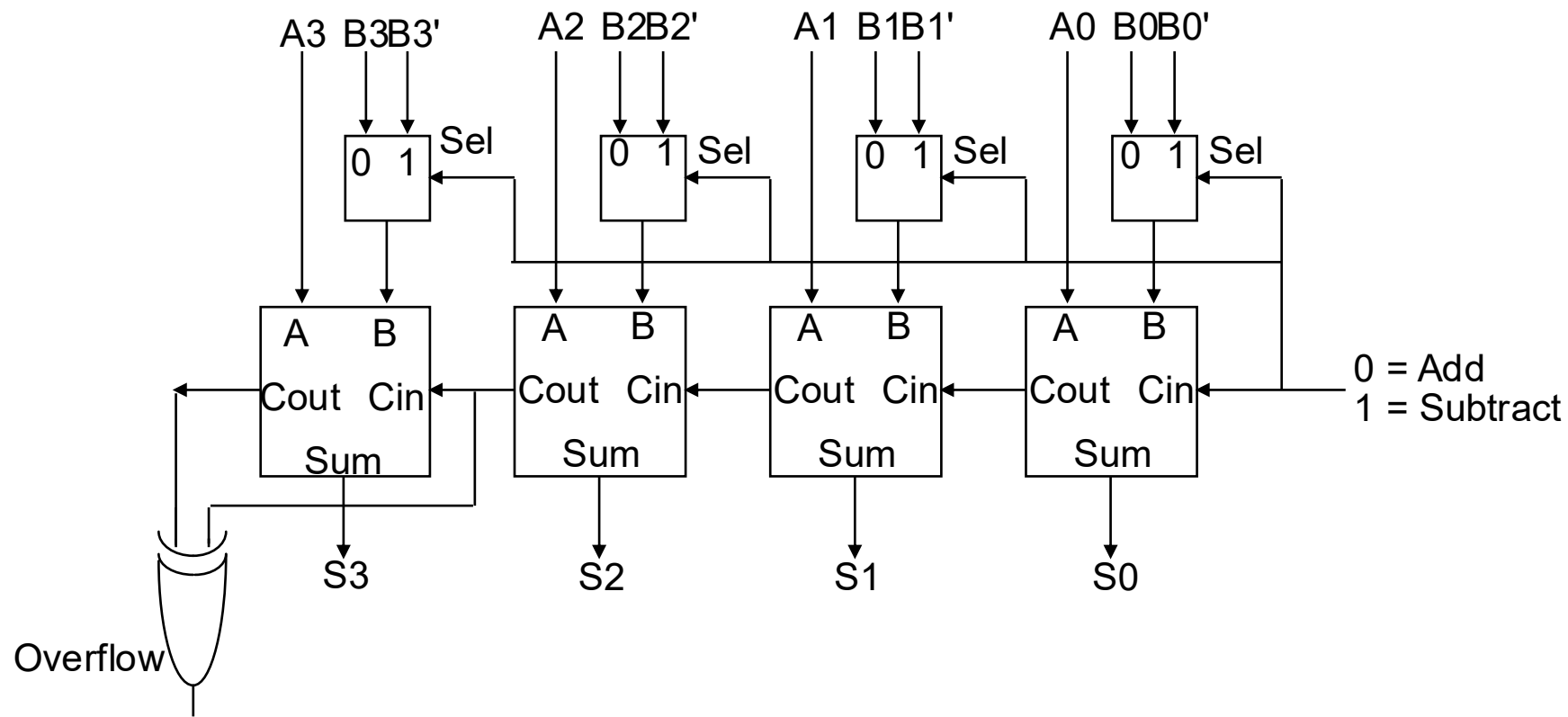
Ripple-Carry Adder: Example

- $3 + 2 = 5$
- $0011 + 0010 = 0101$



Ripple-Carry Adder: Subtraction

- Use adders to subtract with 2s complement representation
 - $A - B = A + (-B) = A + B' + 1$

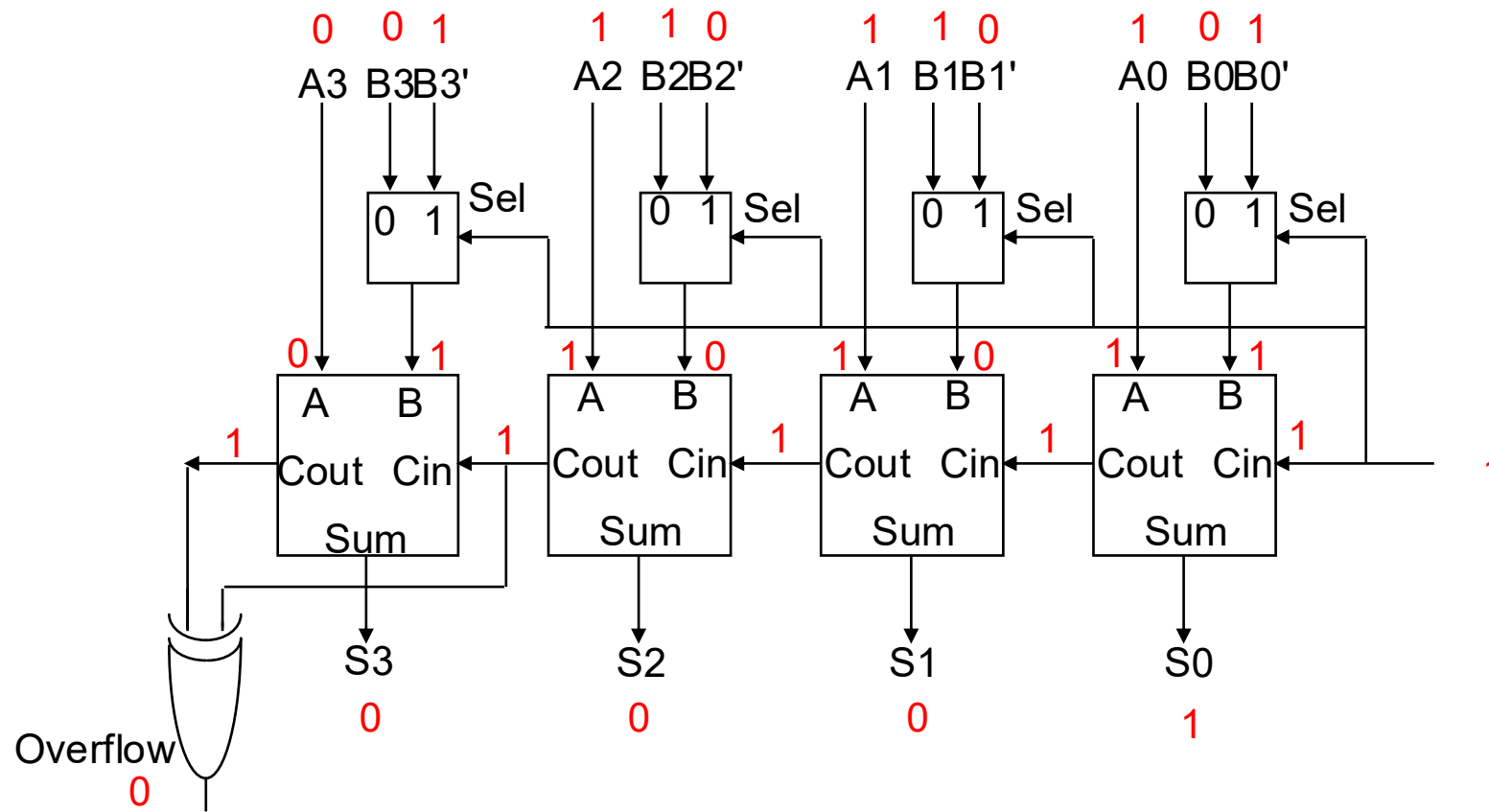


Ripple-Carry Adder: Subtraction

- $7 - 6 = 1$
- $0111 - 0110 = 0001$

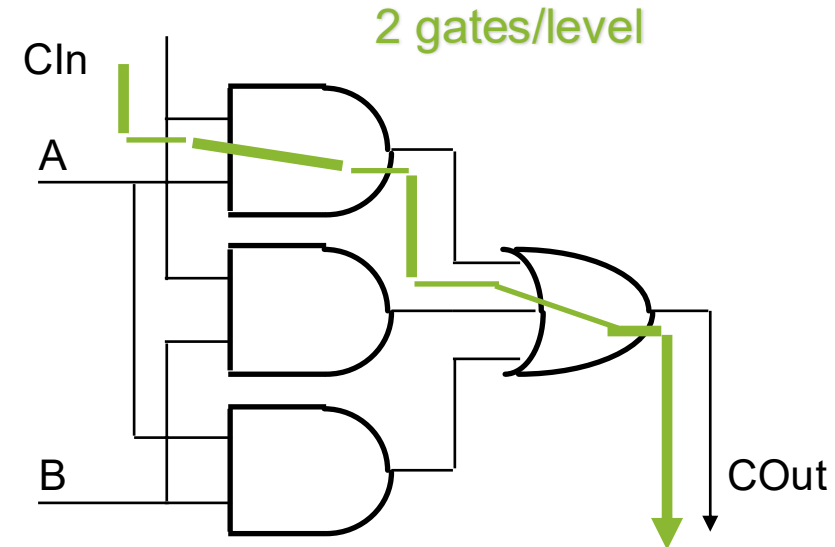
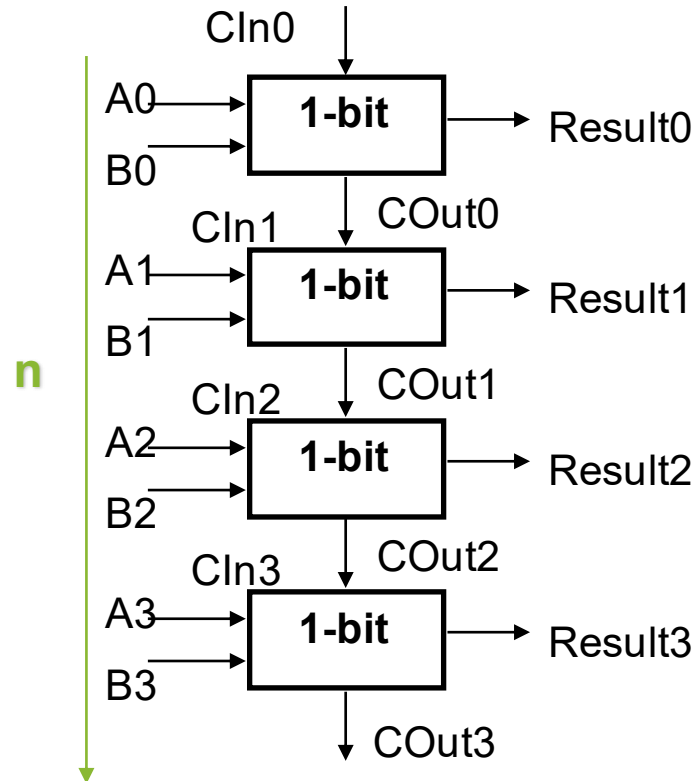
Use adders to subtract with 2s complement representation:

$$A - B = A + (-B) = A + B' + 1$$



Ripple-Carry Adder: Disadvantage

- Long Delay
 - Carry bit may have to propagate from LSB to MSB
 - Worst case delay for n-bit adder = $2n$ gate delays



Ripple-Carry Adder

- Long delay due to the propagation of carry from low to high order stages
- The key to speeding up addition is determining carry values sooner.

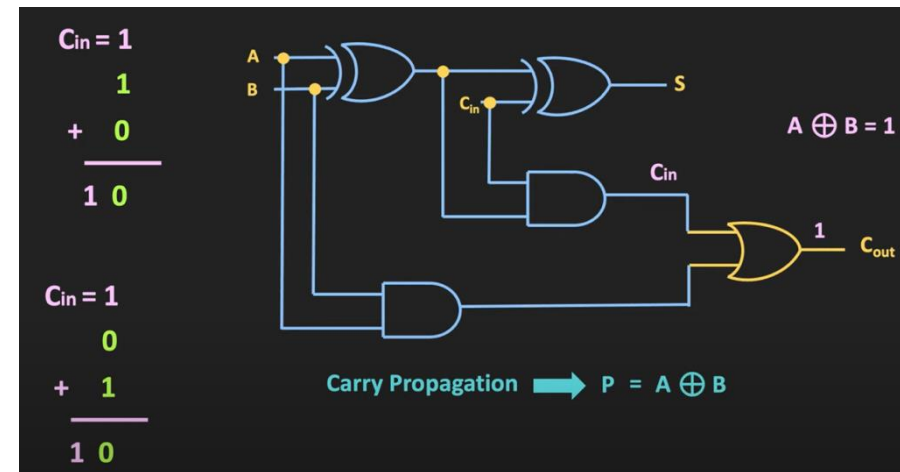
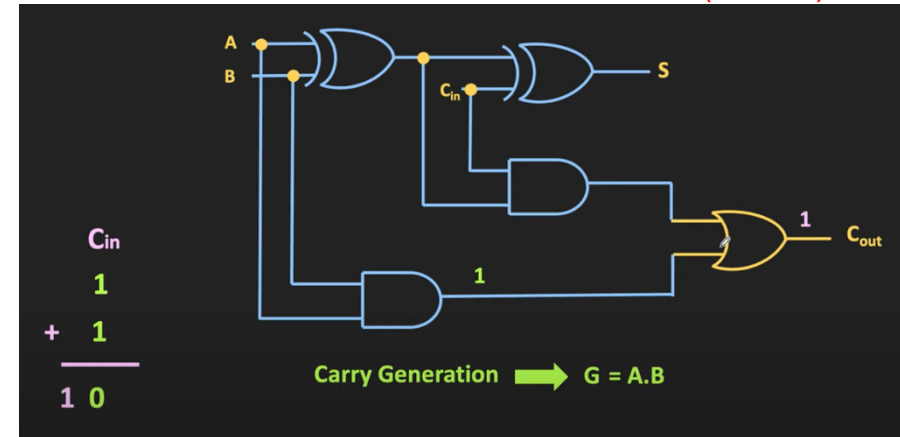
Carry-Lookahead Logic

- Carry generate: $G_i = A_i B_i$
 - Must generate carry when $A = B = 1$
- Carry propagate: $P_i = A_i \text{ xor } B_i$
 - Carry-in will equal carry-out here
- Sum and Cout can be re-expressed in terms of generate/propagate:
 - $\text{Sum}_i = A_i \text{ xor } B_i \text{ xor } C_i$
 $\text{Sum}_i = P_i \text{ xor } C_i$
 - $C_{i+1} = A_i B_i + A_i C_i + B_i C_i$
 $C_{i+1} = A_i B_i + C_i (A_i + B_i)$
 $C_{i+1} = G_i + C_i P_i$

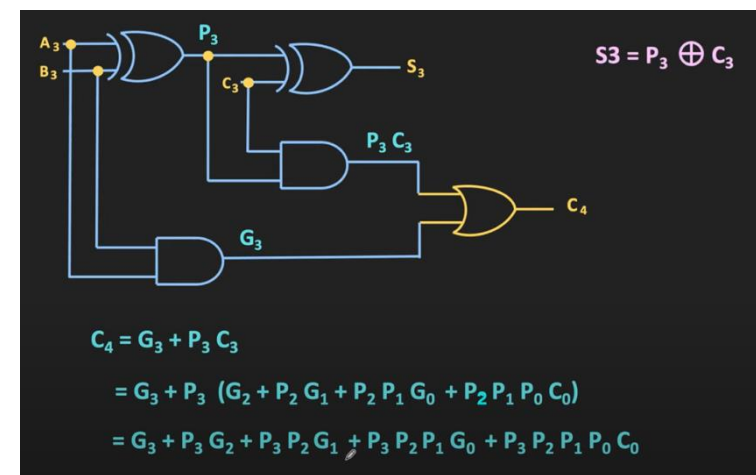
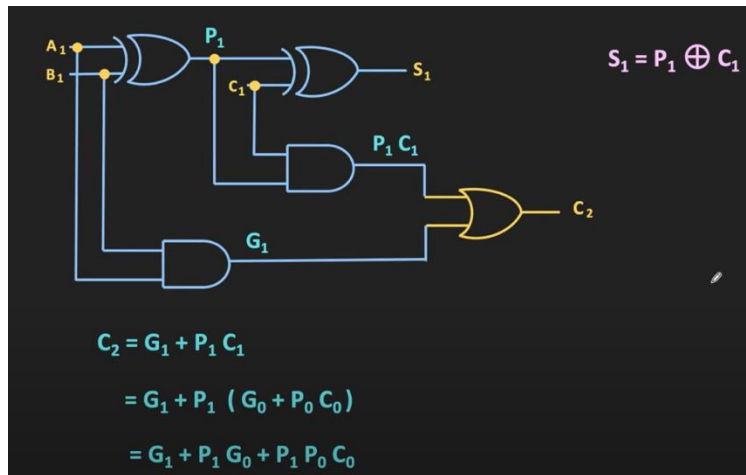
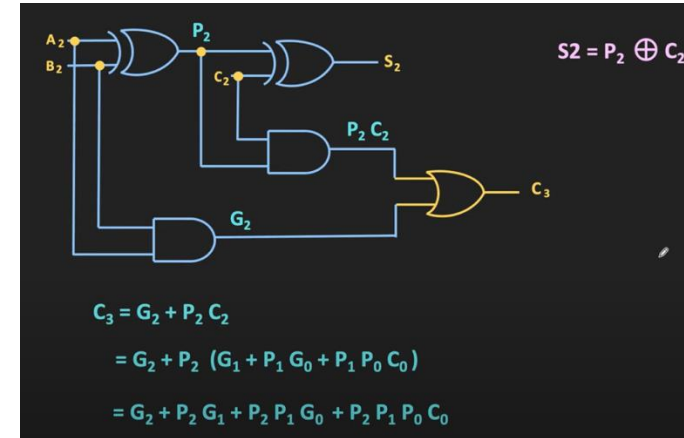
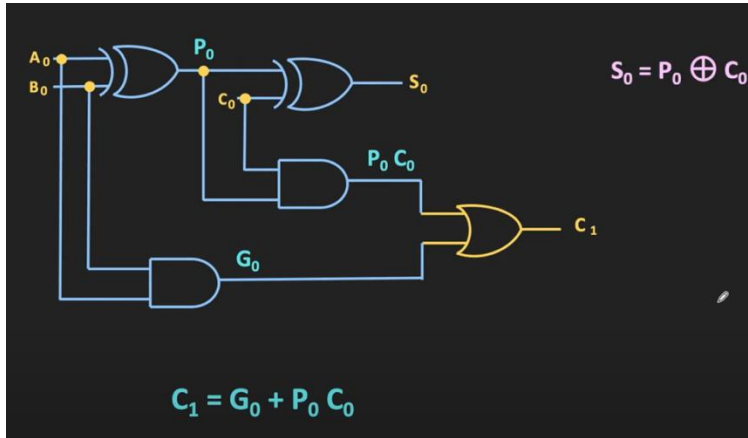
A	B	Cout	Comment
0	0	0	Carry terminate
0	1	Cin	Carry propagate
1	0	Cin	Carry propagate
1	1	1	Carry generate

$$S = A \text{ xor } B \text{ xor } C_{in}$$

$$C_{out} = A B + C_{in} (A \text{ xor } B)$$



Carry-Lookahead Logic

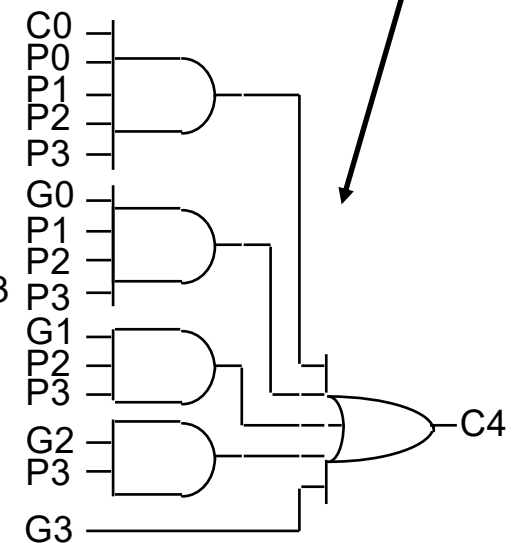
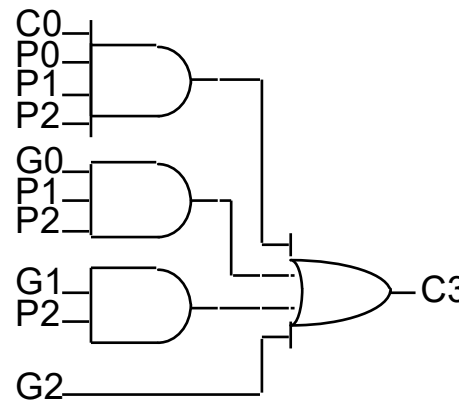
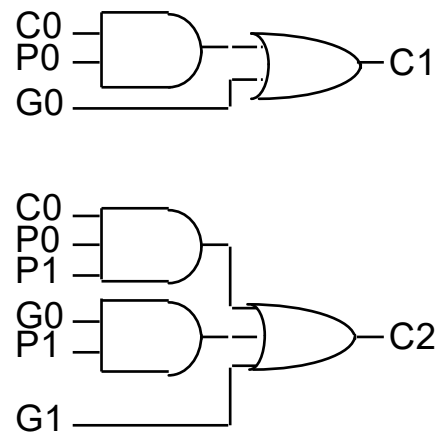
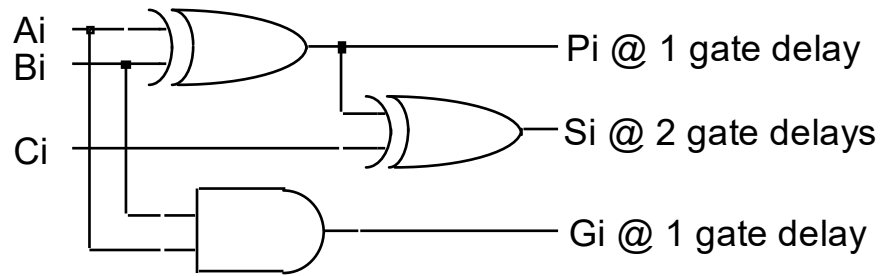


$$G_n = A_n B_n$$

$$P_n = A_n \text{ xor } B_n$$

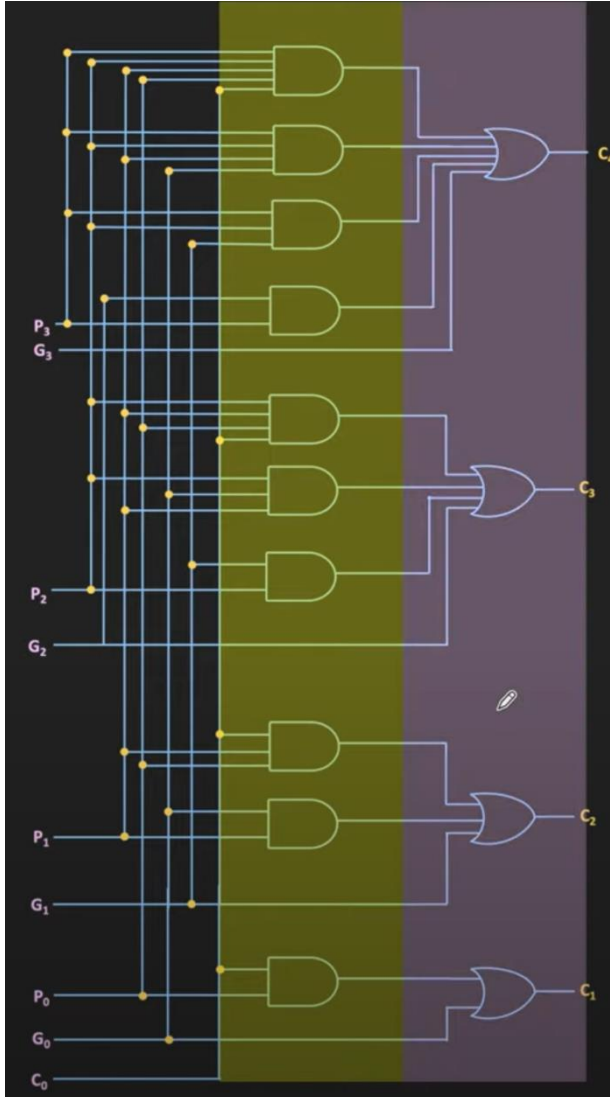
Carry-Lookahead Implementation

- Adder with propagate and generate outputs

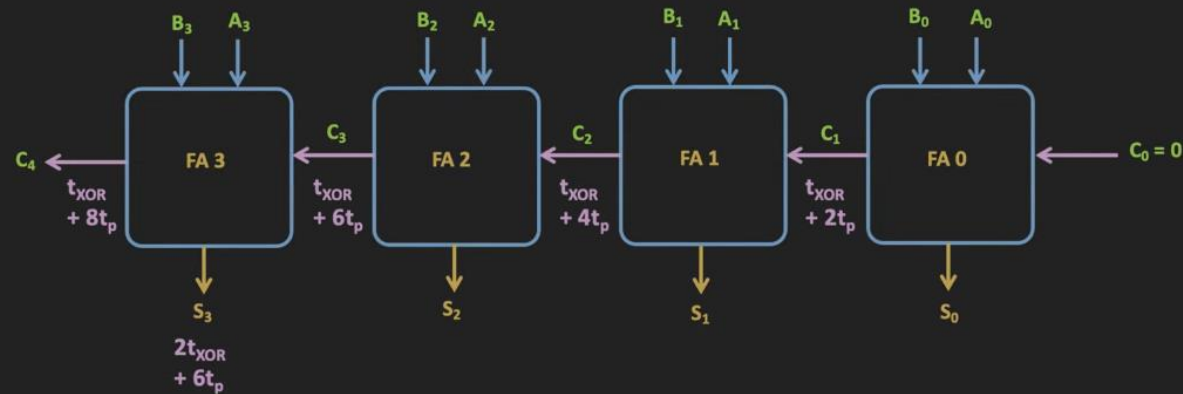


increasingly complex
logic for carries

Carry-Lookahead Implementation



4-bit Ripple Carry Adder



For n-bit adder, If $t_{XOR} = t_p$

Stable Carry output C_{n+1} is available after $t_{XOR} + 2n t_p = (2n + 1) t_p = 9$ Gate Delays

Stable Sum output S_n is available after $2t_{XOR} + 2(n-1)t_p = 2n t_p$

Arithmetic Logic Unit

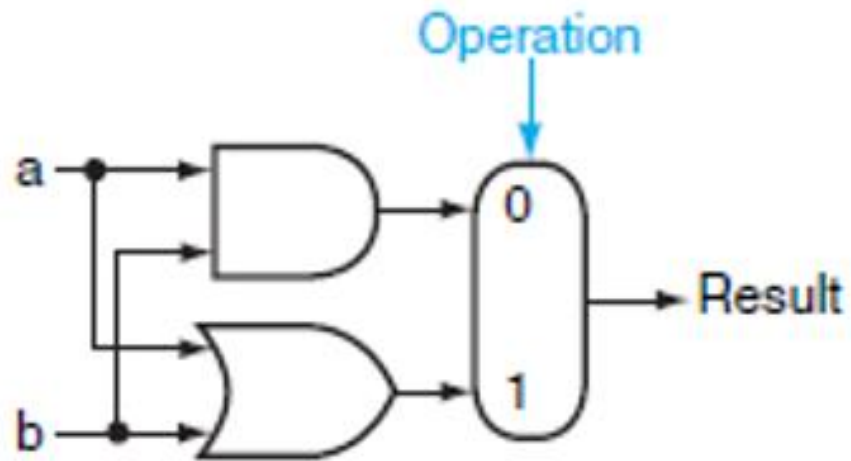
- The Arithmetic Logic Unit (ALU) is the brawn of the computer
 - Performs arithmetic operations (addition, subtraction)
 - Performs logical operations (AND, OR, NOR)
 - Performs logical comparisons (Less Than, Equal To)

Arithmetic Logic Unit

- We will need (at least) a 32-bit ALU
 - Connect 32 1-bit ALUs together

AND, OR

- Uses: AND gate, OR gate, 2:1 mux

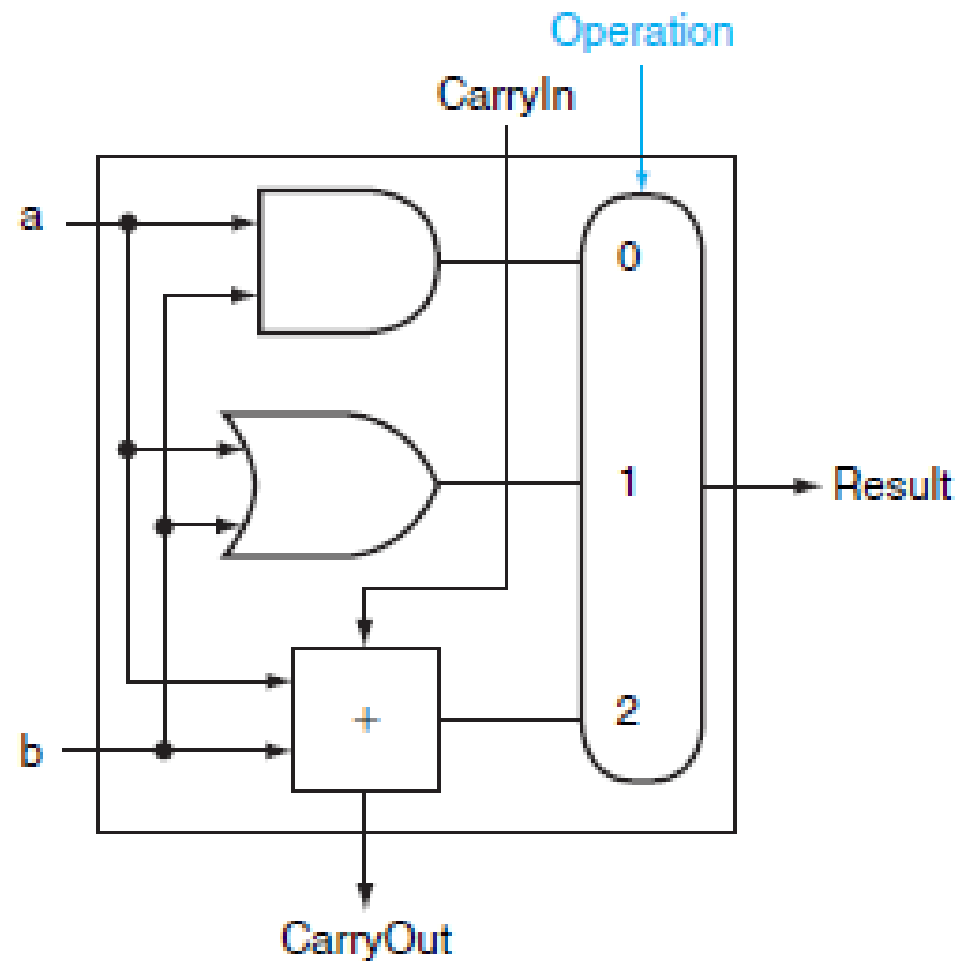


Addition

- Add a 1-bit full adder

Addition

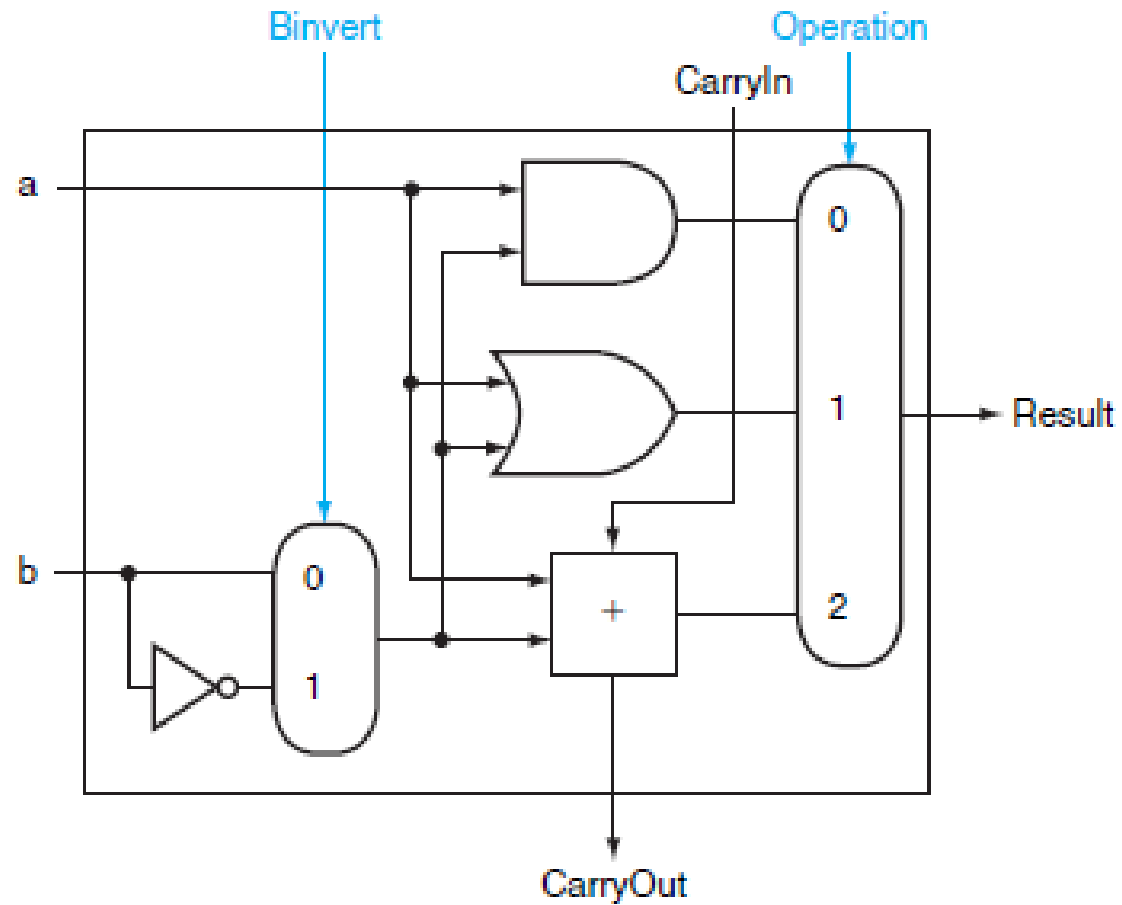
- This 1-bit ALU will perform AND, OR, and ADD



Subtraction

- $(A - B)$ is equivalent to $A + (-B)$
 - Take 2's complement of B and add A
 - 2's Complement: invert every bit and add 1
- To invert B
 - Add a 2:1 mux that chooses between b and b'
- To add 1
 - Set the CarryIn to 1 instead of 0

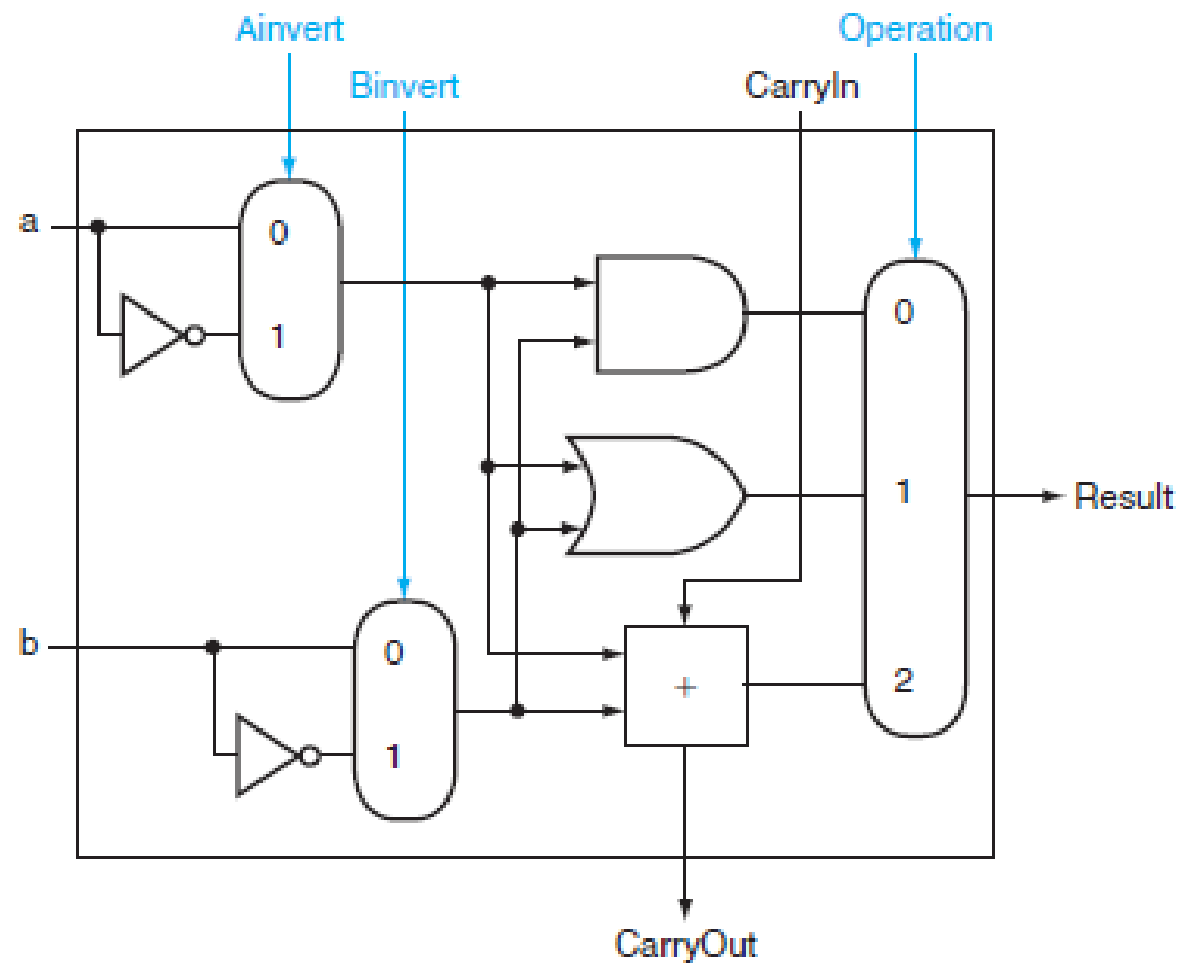
Subtraction



NOR

- $\text{NOR} = \text{NOT} (A \text{ OR } B)$
 $= (A + B)'$
 $= A' * B'$
 $= A' \text{ AND } B'$
- We already have an AND gate and an inverter for B
 - Add an inverter for A

NOR



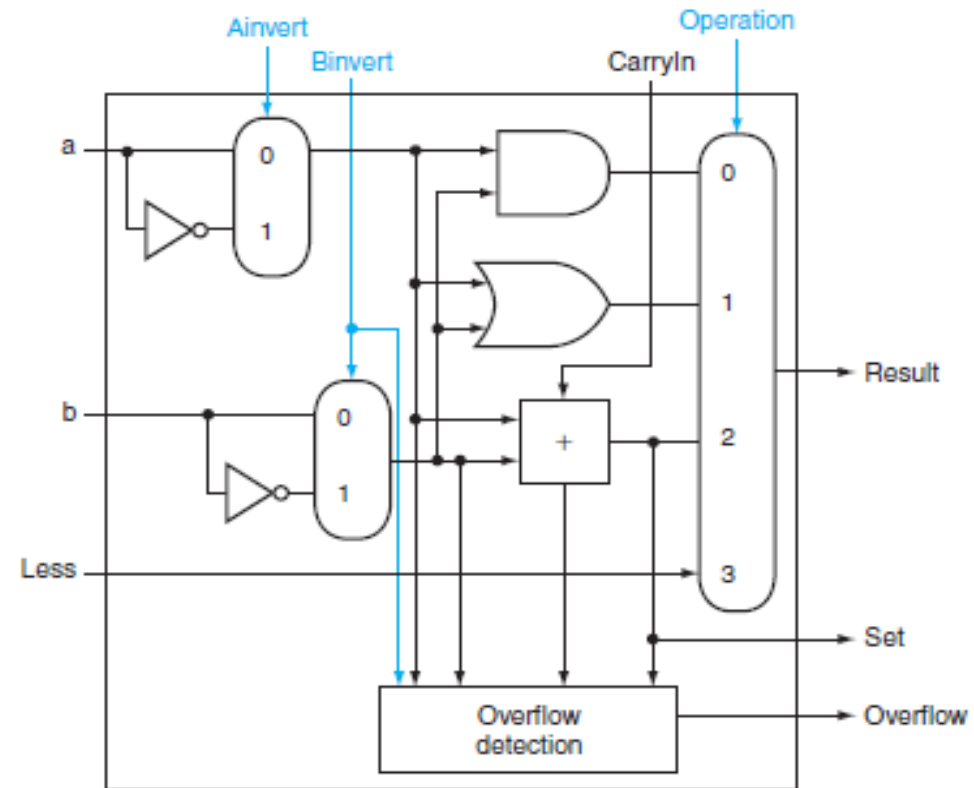
Set on Less Than

- Set on Less Than (slt) is a MIPS instruction
- For inputs A and B it produces 1 if $A < B$, 0 otherwise
- All bits are set to zero except for the least significant
 - Least significant bit determined by the result of $A < B$
- For our ALU to perform this operation we need:
 - A bigger mux
 - Input for the slt (called Less)
 - Method for determining if $A < B$

Set on Less Than

- $A < B$
- $A - B < 0$
 - Subtract and check the sign of the result
 - 1 if less than 0
 - 0 otherwise

Set on Less Than



```

      0 0000 0000
A   = 0001 1001
-B  = 1110 0100
A-B = 1111 1101
    
```

<- This line shows the carries for each column. We can use this to set the overflow output. Since the carry-in and the carry-out for the most significant bit is the same, there is no overflow.

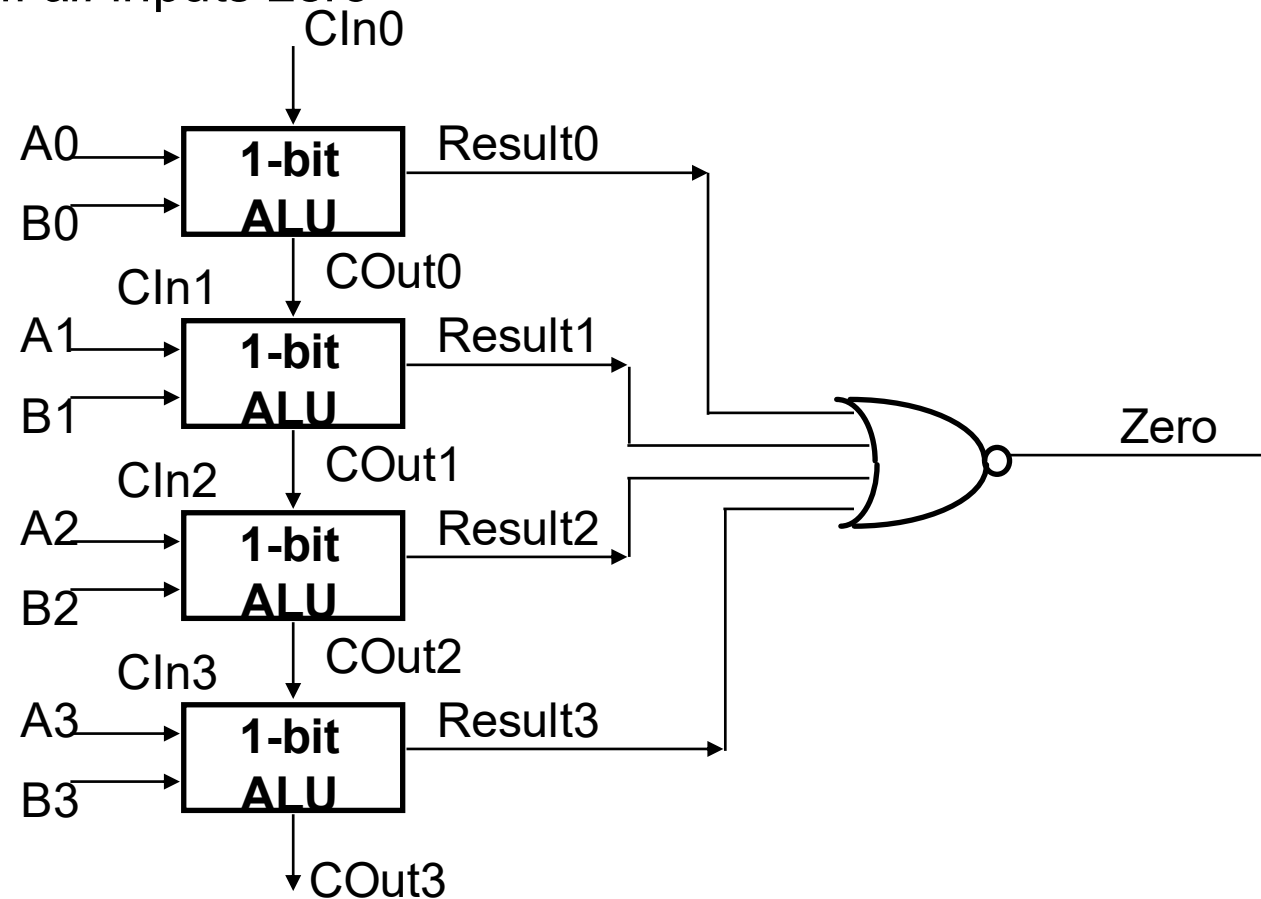
Overflow = 0 Result = 0000 0001

Equal To

- $A = B$
- $A - B = 0$
 - Subtract and check to see if all bits are zero
 - NOR all bits

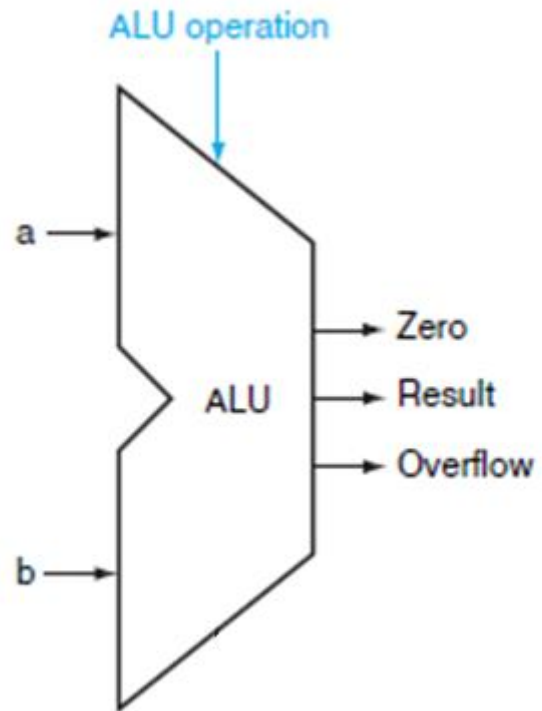
Equal To

- Zero Detection Logic is just a NOR gate
 - Output true (1) only if *all* inputs zero



A 32-Bit ALU

- Universal symbol for ALU





MESUT OZDAG, PH.D.
MESUT.OZDAG@UCF.EDU

